18. Construct a C Program to Simulate Producer Consumer Problem Using Semaphores

Code - Part 1 (Lines 1–35) — Producer Function

```c
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>   // for sleep()
5
6  #define BUFFER_SIZE 5
7  #define MAX_ITEMS 10
8
9  int buffer[BUFFER_SIZE] = {0};
10
11 sem_t empty, full;
12 pthread_mutex_t mutex;
13
14 int produced_items = 0, consumed_items = 0;
15
16 void* producer(void* arg) {
17     while (produced_items < MAX_ITEMS) {
18
19         sem_wait(&empty);
20         pthread_mutex_lock(&mutex);
21
22         // Add item to buffer
23         for (int i = 0; i < BUFFER_SIZE; i++) {
24             if (buffer[i] == 0) {
25                 buffer[i] = produced_items + 1;
26                 printf("Produced: %d\n", buffer[i]);
27                 produced_items++;
28                 break;
29             }
30         }
31
32         pthread_mutex_unlock(&mutex);
33         sem_post(&full);
34
35         sleep(1);
```

Code - Part 2 (Lines 36–68) — Consumer Function

```
36      }
37      return NULL;
38  }
39
40  void* consumer(void* arg) {
41      while (consumed_items < MAX_ITEMS) {
42
43          sem_wait(&full);
44          pthread_mutex_lock(&mutex);
45
46          // Remove item from buffer
47          for (int i = 0; i < BUFFER_SIZE; i++) {
48              if (buffer[i] != 0) {
49                  printf("Consumed: %d\n", buffer[i]);
50                  buffer[i] = 0;
51                  consumed_items++;
52                  break;
53              }
54          }
55
56          pthread_mutex_unlock(&mutex);
57          sem_post(&empty);
58
59          sleep(2);
60      }
61      return NULL;
62  }
63
64  int main() {
65      pthread_t producer_thread, consumer_thread;
66
67      sem_init(&empty, 0, BUFFER_SIZE);
68      sem_init(&full, 0, 0);
```

Code - Part 3 (Lines 69–82) — Main Function

```
69      pthread_mutex_init(&mutex, NULL);
70
71      pthread_create(&producer_thread, NULL, producer, NULL);
72      pthread_create(&consumer_thread, NULL, consumer, NULL);
73
74      pthread_join(producer_thread, NULL);
75      pthread_join(consumer_thread, NULL);
76
77      sem_destroy(&empty);
78      sem_destroy(&full);
79      pthread_mutex_destroy(&mutex);
80
81      return 0;
82  }
```

Output

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 7
Produced: 9
Produced: 10
Consumed: 9
Consumed: 4
Consumed: 6
Consumed: 8
Consumed: 10


...Program finished with exit code 0
Press ENTER to exit console.
```